

# SongEditor: 将零样本歌曲生成语言模型适配为多任务编辑器

Chenyu Yang<sup>1</sup>, Shuai Wang<sup>1,3</sup>, Hangting Chen<sup>2</sup>, Jianwei Yu<sup>2</sup>, Wei Tan<sup>2</sup>, Rongzhi Gu<sup>2</sup>, Yaoxun Xu<sup>4</sup>, Yizhi Zhou<sup>6</sup>, Haina Zhu<sup>5</sup>, Haizhou Li<sup>1,3</sup>

<sup>1</sup> 香港中文大学 (深圳), 中国深圳 <sup>2</sup> 腾讯AI Lab <sup>3</sup> 深圳大数据研究院 <sup>4</sup> 清华大学 <sup>5</sup> 上海交通大学 X-LANCE 实验室 <sup>6</sup> 南京大学新型软件技术国家重点实验室 chenyu yang2@link.cuhk.edu.cn, erichtchen@tencent.com, tomasyu@foxmail.com

## 摘要

新型生成建模范式，特别是音频语言模型的出现，显著推进了歌曲生成领域。尽管最先进的模型能够同时合成长达数分钟的人声和伴奏音轨，但关于对现有歌曲进行部分调整或编辑的研究仍然不足，而这能够实现更灵活有效的制作。在本文中，我们提出了 SongEditor，这是首个将编辑能力引入语言模型歌曲生成方法的歌曲编辑范式，支持片段级和音轨级修改。SongEditor 提供了调整歌词、人声和伴奏的灵活性，以及从头合成歌曲。SongEditor 的核心组件包括音乐 tokenizer、自回归语言模型和扩散生成器，能够生成整个段落、被掩码的歌词，甚至分离的人声和背景音乐。大量实验表明，所提出的 SongEditor 在端到端歌曲编辑中取得了卓越性能，这由客观和主观指标共同证明。

演示 — [https://cypress-yang.github.io/SongEditor\\_demo/](https://cypress-yang.github.io/SongEditor_demo/)

## 引言

近年来，生成人声音乐的应用取得了显著成果。与语音或带伴奏的歌声相比，一首歌曲通常包含多个段落，并穿插着纯音乐片段如前奏、间奏和尾奏，这要求严格的一致性和上下文连贯性。因此，理想的歌曲生成系统必须既结构引导又能处理长内容。

然而，在实践中生成歌曲的某部分可能不尽如人意，但重新生成整个序列需要大量时间和计算资源，并可能导致进一步的问题。因此，歌曲编辑至关重要，因为它允许对音乐作品进行精炼和增强。此外，有时只需要改变特定音轨，如人声或伴奏。如何在不破坏上下文一致性和整体音乐性的情况下重写歌曲的指定部分，是 AI 歌曲创作的一个重大挑战。

图1: SongEditor 支持各种歌曲生成和编辑任务，包括从头创建完整歌曲和填充编辑。Accomp-to-song 和 vocal-to-song 分别表示基于伴奏或人声等部分输入条件生成完整歌曲。

目前，学术界关于端到端歌曲生成的研究相对较少。像 Suno 这样的最先进歌曲生成平台通常用于商业用途且非开源。Jukebox (Dhariwal et al. 2020) 可以合成特定风格和流派的歌曲，但缺乏更精细的控制，如结构调整或特定指令。迄今为止，这些方法主要关注纯生成，无法直接应用于编辑场景。

Ding et al. (2024) 提出了 SongComposer，它支持歌词到旋律和旋律到歌词等多种任务，可能允许某种程度的歌曲编辑。然而，SongComposer 将歌曲分解为歌词和音符 (MIDI)，不保留音色等人声信息。此外，在数据预处理过程中，原始歌曲和提取的 MIDI 文件之间存在不可避免的失真。

在本文中，我们提出了 SongEditor，这是一个通用范式，能够在现有的基于语言模型的歌曲生成方法中实现部分编辑。由于目前可用的歌曲生成模型技术很少，我们首先实现了 SongLM，一个基础的零样本歌曲生成框架，可以基于结构引导的歌词和参考歌曲的 10 秒片段作为声学提示，生成长达两分钟的歌曲段落。随后，我们在架构设计和模型训练过程中进行修改，使系统能够基于不同类型的上下文执行编辑。

如图1所示，除了从头生成歌曲外，SongEditor 还可以重新生成特定时段并分离人声或伴奏。值得注意的是，上述两种能力可以同时执行，甚至可以编辑人声或伴奏的短片段。本文的主要贡献总结如下：

- 我们提出了 SongEditor，一种基于语言模型的生成方法，处理由用户定义歌词和结构引导的长内容生成。SongEditor 是首批不仅能够从头生成歌曲，还能在给定其余部分时对特定片段进行句子级编辑（片段级）以及完成整个人声或伴奏音轨（音轨级）的工作之一。
- 我们采用了一种无上下文编辑方法，能够有效减少对上下文歌词的依赖，并被证明更适合长内容编辑。此外，考虑到伴奏的复杂组成，我们采取了一系列措施来确保编辑片段的一致性和过渡的自然性。
- 我们使用客观指标和人工评估对我们的模型进行了评估。结果表明，我们的模型能够生成具有优越音乐性和出色一致性的歌曲。

## 相关工作

### 零样本语音合成与编辑

零样本文本到语音转换（TTS）能够基于注册录音生成未见说话人的语音。当前主流的生成方法可分为语言模型方法和扩散方法。前者 (Wang et al. 2023a; Du et al. 2024b; Borsos et al. 2023) 提出自回归地生成离散编解码器 token (Defossez et al. 2022)，而后者 (Shen et al. 2023; Chen et al. 2024b; Yang et al. 2024; Vyas et al. 2023; Wang et al. 2023b) 利用扩散模型生成内部连续特征。

此外，一些方法广泛探索了语音编辑任务。例如，VoiceCraft (Peng et al. 2024) 提出了一种用于神经编解码器语言模型中自回归生成过程的 token 重排方法。除此之外，扩散模型 (Vyas et al. 2023; Wang et al. 2023b; Du et al. 2024a) 以非自回归方式天然具备填充编辑能力。这些方法已经实现了高质量的语音重合成。然而，由于这些研究中使用的语音话语相对较短且简单，它们不能直接用于歌曲编辑。

### 音乐生成

最近，音乐生成受到了研究界的广泛关注。与语音相比，音乐通常在音高、旋律、速度和音色方面具有更大程度的变化。为了在无需专家知识的情况下促进用户友好的条件生成，文本到音乐变得异常流行。例如，MusicGen (Copet et al. 2023) 和 Mustango (Melechovsky et al. 2024) 采用 T5 (Raffel et al. 2020) 和 FlanT5 (Chung et al. 2024) 文本编码器处理自然语言提示，能够生成可控音乐。由于配对的音乐和描述数据难以收集，一些方法 (Agostinelli et al. 2023; Lam et al. 2023; Chen et al. 2024a; Evans et al. 2024) 利用跨模态对齐模型 (Huang et al. 2022; Elizalde et al. 2023; Wu et al. 2023) 将音乐和文本都投影到同一嵌入空间。从音乐派生的条件向量可用于以自监督方式训练生成模型，而从文本派生的向量用于实际合成目的。一些方法将音乐信号（如 chromagram (Copet et al. 2023) 或节拍 (Lin et al. 2023)）作为辅助条件。这些附加信息有助于使生成过程更可控。

一些有趣的研究也关注音乐编辑。例如，Music ControlNet (Wu et al. 2024a) 接收多种类型的音乐信号以实现时变控制。MusicMagus (Zhang et al. 2024) 旨在通过比较新旧提示之间的差异来部分编辑生成音乐的特定属性。同时，VampNet (Garcia et al. 2023) 被设计为用非自回归 Transformer 完成声学 token 序列中的被掩码区域。

### 歌声合成

歌声合成（SVS）旨在基于歌词和乐谱生成成人声。受益于音符音高和时长，SVS 过程变得更加可控且更容易收敛。

传统的 SVS 系统主要关注将音素级旋律控制纳入生成的声学表示中。例如，VISinger (Zhang et al. 2022a,b) 采用类似于 VITS (Kim, Kong, and Son 2021) 的方法，学习声学单元和文本单元之间的对齐。TokSing (Wu et al. 2024b) 使用多个自监督学习模型将原始音频离散化，并在训练和推理阶段进一步增强旋律信息。最近的模型 (Liu et al. 2022; Hwang, Lee, and Lee 2025; He et al. 2023) 在高品质歌声生成方面展示了有前景的性能。例如，HiddenSinger (Hwang, Lee, and Lee 2025) 利用潜在扩散模型 (Rombach et al. 2022) 合成正则化的中间编解码器表示。同时，RMSSinger (He et al. 2023) 提出了一种基于 DDPM 的方法 (Ho, Jain, and Abbeel 2020)，使用词级粗粒度乐谱合成成人声，减轻了现实场景中的人工标注的沉重需求。

## 基础模型：SongLM

在介绍编辑框架之前，我们首先展示基础系统 SongLM，作为我们编辑范式的基础。图2展示了 SongLM 的架构框架，它接受多个歌词句子和一个 10 秒声学提示作为输入。该框架的主要组件包括语义 tokenizer、语言模型和基于扩散的生成器。语义 tokenizer 将音频波形（包括声学提示、上下文和分离音轨）压缩为离散语义 token。随后，语言模型以自回归方式生成语义 token。最终波形由扩散生成器从输出序列重建。

训练过程分为两个不同阶段。首先，tokenizer 和扩散生成器同时训练。随后，语言模型使用 tokenizer 生成的 token 进行训练。在第二阶段，tokenizer 和扩散生成器均保持冻结状态。

**歌词预处理器：**原始数据对仅包含纯文本和波形  $\{(X_i, W_i)\}_{i=1}^L$ ，其中  $L$  是歌词数量， $X_i$  是第  $i$  句， $W_i$  是对应波形。值得注意的是，波形和歌词都是连续的，因此它们可以按顺序拼接，且  $W_i$  在非人声段落可能为空。然而，仅依赖歌词文本不足以完成复杂的歌曲生成任务。为了将结构信息纳入歌词，我们应用了结构检测器 (Kim and Nam 2023)。结构指示符的类别见附录A。

**语义 Tokenizer：**神经编解码器 token 已被先前的音乐生成系统 (Huang et al. 2022) 广泛使用。然而，我们发现从自监督训练模型导出的量化语义 token (Borsos et al. 2023; Agostinelli et al. 2023) 表现出更高的压缩率。在本文中，我们提出了一种由两个不同分支组成的 tokenizer，使用不同的预训练编码器：MERT (Li et al. 2024) 和 HuBERT (Hsu et al. 2021)。前者关注伴奏，后者用于人声。两个残差向量量化器 (RVQ) (Zeghidour et al. 2021; Kumar et al. 2023) 被附加在每个分支的末端。每个量化器有两层，导致每帧产生  $K = 4$  个 token。语义 token 提取可以表述为：

$$Y = \text{RVQ}(\text{MERT}(W)) \oplus \text{RVQ}(\text{HuBERT}(W)), \quad (1)$$

其中  $Y \in \mathbb{Z}^{T \times K}$  是离散 token 序列， $T$  是序列长度。 $W$  是输入波形。 $\oplus$  表示逐帧拼接。在训练阶段，仅使用承诺损失 (van den Oord, Vinyals, and kavukcuoglu 2017) 更新 RVQ 层：

$$\mathbb{L}_{rvq} = \sum_k (\|sg(e_k) - z_k\|_2^2 + \|e_k - sg(z_k)\|_2^2), \quad (2)$$

其中  $sg(\cdot)$  是停止梯度操作， $z_k$  是输入潜在向量， $e_k$  是最近的码本条目。

**解码器-only 语言模型：**条件张量和语义 token 序列首先拼接在一起，然后馈入解码器-only Transformer。每个 Transformer 层包含带有旋转位置嵌入 (RoPE) (Su et al. 2024) 的因果自注意力机制和前馈块。

**扩散生成器：**我们利用潜在扩散模型 (LDM) (Rombach et al. 2022) 作为生成器，包括扩散模型、变分自编码器 (VAE) (Kingma and Welling 2013) 和声码器。我们将传统的 U-Net (Ronneberger, Fischer, and Brox 2015) 主干替换为 DiT (Peebles and Xie 2023)，它以语义 token 为条件，对 VAE (van den Oord, Vinyals, and kavukcuoglu 2017) 产生的潜在向量应用前向和反向过程。VAE 随后将潜在向量转换为梅尔频谱图，再通过 HifiGAN (Kong, Kim, and Bae 2020) 声码器转换为波形。

## SongEditor

在本节中，我们将介绍 SongEditor，一个将编辑能力集成到基础模型中的新颖框架。SongEditor 支持两种类型的歌曲编辑：片段级编辑，同时修改歌曲片段内的人声和伴奏；音轨级编辑，允许独立编辑人声或伴奏。重要的是，这些方法可以组合，赋予在单个音轨或整体上编辑歌曲片段的灵活性。下面，我们将详细说明 SongEditor 如何实现这些编辑功能。

图2：所提出的 SongEditor 框架的架构。我们首先联合训练 DiT 和 RVQ，然后训练语义语言模型。多源编码器仅用于音轨级编辑。

### 长内容片段级编辑

对于片段级编辑，我们首先讨论利用被编辑片段的上下文信息。然后，我们介绍一种类似于 VoiceCraft (Peng et al. 2024) 的重排操作，应用于语义 token 序列。为了实现更自然的过渡效果，我们提出了训练期间的力平滑策略和推理期间基于分数的候选选择。

**上下文选择：**待编辑的波形片段定义为  $W_{[L_A:L_B]} = [W_{L_A}, W_{L_A+1}, \dots, W_{L_B}]$ ，其中  $1 \leq L_A \leq L_B \leq L$ 。这里， $L_A$  和  $L_B$  表示起始和结束句子。值得注意的是， $L_A$  和  $L_B$  可以等于 1 或  $L$ ，允许 SongEditor 也执行续写或生成任务。在训练阶段，前文和后文  $y_{[1:A]}$  和  $y_{(B:T]}$  直接从 token 序列  $Y$  中切出，其中  $A$  和  $B$  是帧的起始和结束索引。在推理期间，这些上下文由语义 tokenizer 分别从波形  $W_{[1:L_A]}$  和  $W_{(L_B:L]}$  中提取，重叠 1 秒以避免边缘效应。重叠区域在 tokenization 后移除。对于歌词输入，仅保留被编辑的句子，表示为  $X = [X_{L_A}, X_{L_A+1}, \dots, X_{L_B}]$ ，我们称之为歌词无上下文策略。不包含歌词上下文的选择基于两个观察：1) 歌词句子之间的上下文联系并不特别紧密。2) 无上下文方法可以实现对重合成成人声更精确的控制，因为上下文歌词有时会导致错误的人声生成。

此外，我们观察到无上下文策略更适合实际应用，例如对于超过训练样本最大长度的歌曲进行连续生成。通过消除对上下文歌词的需求，语言模型可以自动逐步生成更长的歌曲 (Agostinelli et al. 2023)，而无需在每一步手动截取和标注前文上下文。这种方法提高了实际场景中的效率和可用性。

**重排：**假设原始序列由三个片段组成： $Y = [y_{[1:A]}, y_{[A:B]}, y_{(B:T)}]$ ，在训练阶段，编辑片段被移动到序列末尾，并在每个片段末尾附加一个  $\langle \text{SEP} \rangle$  token。因此，重排后的序列应为  $Y^{re} = [y_{[1:A]}, y_{\langle s \rangle}, y_{(B:T)}, y_{\langle s \rangle}, y_{[A:B]}, y_{\langle s \rangle}]$ ，其中  $y_i = (y_{i,k})_{k=1}^K$ ， $y_{\langle s \rangle}$  表示  $\langle \text{SEP} \rangle$  token 的嵌入。

进一步应用延迟模式 (Copet et al. 2023) 重排  $K$  个 RVQ token。第  $k$  个量化器的 token 向后移动  $(k-1)$  个时间步，并在每个片段末尾附加额外的  $(K-1)$  个空帧以防止量化器重叠。然后同一时间步的 token 被堆叠在一起。因此，语言模型输入的最终长度应为  $(T+3K)$ 。由于上下文的歌词已被丢弃，仅计算编辑片段的训练损失： $\mathbb{L}_{ce} = -\log(P_\theta^{[A,B]})$ ，其中

$$\begin{aligned} P_\theta^{[A,B]} &= P_\theta(y_{[A:B]} | y_{[1:A]}, y_{(B:T)}, X, Y^{stly}) \\ &= \prod_{i=A}^B \prod_{k=1}^K p_\theta(y_{i,k} | \forall y_{m,n}, X, Y^{stly}), \end{aligned}$$

如果  $m \in (0, A) \cup (B, T)$  或  $n+m < i+k$ 。

**力平滑训练：**由于旋律和节奏的存在，听众对音乐中的中断往往比语音更敏感和严格。然而，确保平滑性对音乐编辑更具挑战性，尤其是在编辑片段的端点处。为解决此问题，我们提出了力平滑策略。在训练阶段，模型被强制在编辑片段结束后仍以 0.1 的概率预测额外的  $\lambda$  帧。这些帧直接从后续上下文  $y_{(B:T)}$  的开头复制，用于损失计算的  $P_\theta^{[A,B]}$  被替换为  $P_\theta^{[A,B+\lambda]}$ 。在推理期间，模型首先贪婪地确定是否在当前步停止预测。只要  $\langle \text{es} \rangle$  的概率最大，模型将立即输出  $\langle \text{es} \rangle$  token。否则，将采用 top-k 采样方法从剩余选项中随机选择下一个 token，这种采样策略可以有效缓解训练期间力平滑导致的过度写入问题。

**基于分数的候选选择：**为了实现更好的过渡，推理期间应用了基于分数的候选选择 (Jiang et al. 2023)。具体来说，我们首先生成一个初始候选  $\hat{y}_{[A:B]}^1$ ，并重新合成最后 3 秒  $(N-1)$  次，得到一组  $N$  个候选  $\hat{Y}_{[A:B]} = \{\hat{y}_{[A:B]}^1, \hat{y}_{[A:B]}^2, \dots, \hat{y}_{[A:B]}^N\}$ 。每个候选被用作新前缀来预测后续  $\lambda$  帧。每个候选的分数可以表示为  $y_{(B:B+\lambda)}$  的对数似然。最终选择分数最高的候选，可以表述为：

$$y = \arg \max_{\hat{y} \in \hat{Y}_{[A:B]}} P_\theta(y_{(B:B+\lambda)} | y_{[1:A]}, \hat{y}, X, Y^{stly}).$$

## 多源音轨级编辑

此外，我们探索了将人声和伴奏补全集成到模型中。如图2所示，SongEditor 利用分离的人声或伴奏音轨作为辅助上下文来补全另一个。提供的音轨通过门控多源编码器处理，并纳入语言模型。下面介绍此过程的细节。

**源分离：**采用带分离 RNN (BS-RNN) (Yu et al. 2023; Luo and Yu 2023) 作为音轨级编辑的源分离模块。BS-RNN 将频谱图分割成多个子带，并在子带和帧上执行双向 LSTM 层。鉴于音乐通常具有更高的采样率和更宽的频率范围，BS-RNN 特别适合音乐源分离。我们利用 BS-RNN 从混合中分离人声，将剩余部分作为伴奏。

在训练阶段，向分离的人声添加白噪声  $\epsilon \sim \mathcal{N}(0, \sigma^2)$ ，以防止分离后剩余音乐成分的潜在泄漏 (Donahue et al. 2023)。在本文中， $\sigma$  设置为 0.01，导致信噪比 (SNR) 为 40dB。

**多源编码器：**如图2所示，为了兼容各种条件，人声和伴奏共享相同的 tokenizer 和嵌入层。每帧的 token 嵌入可以表示为：

$$c_i = \begin{cases} \sum^K c_{i,k} + t & t \in \{t^M, t^V\} \\ t & t = t^\odot. \end{cases} \quad (6)$$

其中  $t^M$ 、 $t^V$  和  $t^\odot$  分别是表示源类型（伴奏、人声和无）的特殊嵌入。 $c_{i,k} \in \mathbb{R}^D$  是量化器嵌入， $D$  是嵌入向量的维度。如果提供一个源，序列应为  $c = (c_t)_{t=1}^T \in \mathbb{R}^{T \times D}$ ；否则， $c \in \mathbb{R}^{1 \times D}$ 。该序列通过带有 RoPE 的多层 Transformer 编码器，并通过交叉注意力注入每个 Transformer 解码器层。

## 实验设置

### 数据集

对于 SongEditor 的训练，使用了约 70 万首歌曲的大规模数据集，总计 5 万小时。采用类似于 Yu et al. 2024) 的数据清洗流程来过滤低质量样本并在预处理过程中校正歌词时间戳。最初使用 BSRNN 提取人声。然后分别利用 Pyanote (Bredin 2023) 和 WhisperX (Bain et al. 2023) 工具包进行语音活动检测 (VAD) 和歌声识别 (SVR)。严重不匹配的歌词将被丢弃。时间边界将根据 VAD 结果进行校正，相邻的两个歌词句子如果间隔很短将被合并。

对于客观评估，我们组装了一个包含 200 个随机选择样本的测试集。每个样本长度在 90 到 120 秒之间，并包含至少一个完整的主歌和副歌。从歌曲的其他部分提取 10 秒提示。

表1：基线和我们提出的方法在歌曲生成上的评估结果。

| 方法          | PER(%)↓ | FAD↓ | Musicality↑ | Quality↑ |
|-------------|---------|------|-------------|----------|
| GT(restore) | 5.51    | 0.86 | 3.63        | 3.59     |
| SongLM      | 20.63   | 1.99 | 2.95        | 3.06     |
| SongEditor  | 18.33   | 2.24 | 3.02        | 3.19     |

对于歌曲编辑任务，我们随机掩码一个区域，该区域可能位于歌曲的中间、开头或结尾。对于主观评估，我们随机选择 15 个样本并手动校正标注，每个样本生成两个录音，然后由专家音乐人进行评估。

### 评估指标

为了进行全面比较，我们评估了三种不同的模型：SongLM、SongEditor 和 SongEditor+。SongLM 作为我们的基线，只能从头生成歌曲。SongEditor 支持片段级编辑，而 SongEditor+ 能够进行片段级和音轨级编辑。它们的详细信息见附录B。

所提出的模型在客观和主观指标方面均进行了评估。对于客观评估，采用了音素错误率 (PER) 和 Fréchet 音频距离 (FAD) (Kilgour et al. 2019)。为了计算 PER，首先通过 BSRNN 提取人声。然后使用 Whisper-large-v2 进行语音识别。歌词中的所有标点和结构 token 已被移除。FAD 分数基于 MERT-95M 最后一层的内部表示计算，使用 fadtk2 工具包计算统计量和比较。对于编辑任务，仅考虑编辑片段。

对于人工评估，我们进行了平均意见分数 (MOS) 听力测试。具体来说，我们采用 30 名听众对每个音频样本在 1 到 5 分之间进行评分，涵盖五个方面：音乐性、质量、连贯性、平滑度和可懂度。连贯性衡量编辑部分与上下文之间的一致性程度。平滑度指过渡的自然性。这两个分数仅对片段级编辑任务考虑。

在我们的实验中，我们观察到从原始音频恢复的音轨有时会显著降低源分离模块的性能，导致异常高的 PER。为了更精确地调整人声质量，我们还要求听众评估生成的人声是否清晰且与给定歌词匹配（可懂度），用于音轨级编辑任务。

# 结果与分析

## 歌曲生成

表1比较了我们提出的方法与基线在歌词到歌曲生成方面的性能。此外，我们展示了从真实语义 token 直接恢复音频 ("restore") 的结果，这代表了该框架的上限。加入片段级编辑能力并未降低 SongLM 的性能。相反，在主观 MOS 分数和 PER 方面可以观察到轻微改善。

## 片段级编辑

表2报告了 SongEditor 在片段级编辑中的结果。为了展示我们模型中使用的无上下文策略的优势，我们采用 VoiceCraft（一种基于上下文的语音编辑方法）作为基线。VoiceCraft 将两个上下文和修改句子的完整歌词作为输入。在训练期间，起始点和结束点是随机选择的。所提出的 SongEditor 在 PER 上显著优于基线。我们进一步发现，句子级错误，如重复或遗漏，更容易发生在编辑片段附近。我们认为这些错误是由于未能对齐上下文音频和歌词的边界造成的。在大多数主观指标中，特别是连贯性分数，SongEditor 也超越了基线，这证明无上下文策略更适合这种长内容编辑任务。一个值得注意的观察是，我们提出的模型的平滑度分数下降了。我们将这种下降归因于训练数据中时间边界标注的潜在不准确性。

我们进行了一项消融研究来验证力平滑训练的有效性。我们重新训练了 SongEditor 相同的步数，但不使用力平滑。如表2所示，所有指标都有不同程度的下降。这可能表明力平滑训练不仅使模型能够产生更平滑的过渡，还帮助它学习后续上下文中的长期依赖关系。此外，在推理期间使用基于分数的候选选择，SongEditor 即使没有进一步训练也能获得更高的平滑度分数。

表2：SongEditor 在片段级编辑中的评估结果。"CS"表示基于分数的候选选择。由于它仅影响局部平滑度，其他指标被省略。"FS"表示力平滑。

| 方法         | Context-Free | PER(%)↓ | FAD↓ | Musicality↑ | Quality↑ | Coherence↑ | Smoothness↑ |
|------------|--------------|---------|------|-------------|----------|------------|-------------|
| VoiceCraft | ×            | 35.23   | 1.34 | 3.03        | 3.17     | 3.48       | 3.63        |
| SongEditor | ✓            | 20.82   | 1.30 | 3.25        | 3.39     | 3.82       | 3.53        |
| -CS        | ✓            | -       | -    | -           | -        | -          | 3.50        |
| -CS & FS   | ✓            | 25.68   | 1.37 | 3.13        | 3.23     | 3.54       | 3.43        |

图3比较了过渡区域的梅尔频谱图。我们随机生成了几个具有相同上下文的样本。由于自回归生成从左到右进行，从前文上下文到编辑片段的过渡相对简单，因此我们主要关注从生成到后续音频的过渡。在每个样本的终点周围截取 10 秒音频片段。如图3所示，在应用力平滑和候选选择后，生成样本的梅尔频谱图看起来更平滑，高能量带变得更加连续。相比之下，不使用力平滑时，在过渡点可以观察到明显的不连续或不匹配。

图3：过渡的梅尔频谱图。红框中心是过渡点。左半部分是生成的，右半部分是从真实值恢复的。

## 音轨级编辑

此外，我们评估了具有交叉注意力层的 SongEditor 在音轨级编辑任务中的表现。在本节中，我们主要使用可懂度 MOS 分数来评估生成人声的正确性，因为组合恢复和生成的音轨可能导致异常 PER 结果。我们的实验通过测试各种配置来研究不同源音轨的影响：仅人声 (vocal-to-song)、仅伴奏 (accomp-to-song) 以及两者都不提供。音轨级信息被集成到生成和片段级编辑任务中。

如表3所示，人声和伴奏都在音乐性和 FAD 分数方面提升了性能。人声音轨在音轨级编辑中起主导作用，这是预期的，因为人声通常作为歌曲的骨架，直接传达节奏和情感等关键元素。此外，听众自然更关注人声。相反，从伴奏生成人声面临更大挑战，因为模型必须保持与伴奏的精确节奏同步，导致可懂度分数略有下降作为权衡。值得注意的

是，从头开始 vocal-to-song 生成的 FAD 分数相当高（2.71），但当提供上下文信息时显著下降（1.13）。这表明模型在上下文可用时可以学习更详细的伴奏特征。

表3：SongEditor+ 在不同源音轨下的性能。"V"是人声的缩写，"A"是伴奏的缩写。上半部分比较纯生成的性能，下半部分引入额外的上下文信息。

| 输入<br>V | 输入<br>A | w/<br>context | PER(%)↓ | FAD↓ | Musicality↑ | Quality↑ | Coherence↑ | Smoothness↑ | Intelligibility↑ |
|---------|---------|---------------|---------|------|-------------|----------|------------|-------------|------------------|
| ✓       | ×       | ×             | -       | 2.71 | 3.38        | 3.54     | -          | -           | 4.35             |
| ×       | ✓       | ×             | -       | 1.29 | 2.83        | 2.90     | -          | -           | 3.74             |
| ×       | ×       | ×             | 19.30   | 2.61 | 2.68        | 2.92     | -          | -           | 3.98             |
| ✓       | ×       | ✓             | -       | 1.13 | 3.56        | 3.66     | 4.19       | 3.99        | 4.33             |
| ×       | ✓       | ✓             | -       | 1.17 | 3.21        | 3.06     | 3.48       | 3.72        | 3.53             |
| ×       | ×       | ✓             | 23.72   | 1.32 | 2.93        | 3.06     | 3.33       | 3.47        | 3.57             |

图4展示了参考音频和生成音频的音乐分离结果。我们的目标是研究重新生成的音频与原始源输入的一致性。具体来说，我们首先将参考音频分离为人声和伴奏，然后使用 SongEditor+ 独立完成每个音轨。如图所示，生成的人声频谱图在 vocal-to-song 任务中与输入非常相似。相反，对于 accomp-to-song，虽然恢复的 chromagram 总体上保持一致，但在 chromagram 和频谱图中都可以观察到一些差异。我们认为人声相对独立且更容易压缩，而伴奏通常依赖人声并包含更多细节信息。

图4：参考音频和生成音频的音乐分离结果。

连续与多歌手故事模式

如第4节所强调，无上下文策略在实际应用中展现出显著优势。在本节中，我们探索通过迭代的逐轮过程生成超过两分钟的完整歌曲，称为故事模式，遵循 Agostinelli et al. (2023)。完整歌词被预先分成几个部分。在每一轮中，仅提供当前部分的歌词，同时从先前输出自动提取固定长度的 token 序列作为前缀。整个过程无需人工干预。

具体来说，我们采用两种任务设置：完整歌曲生成和多歌手故事模式。前者旨在从前奏到尾声生成完整歌曲，不改变风格提示。该任务关注各轮之间的连贯性，我们以 60 秒的步长推进。对于后者，我们使用两个不同的风格提示，通常一个用于男声，一个用于女声。主歌和副歌部分使用交替提示生成。为了防止前缀中出现人声，每轮结束时生成一个短器乐片段，步长设置为 5 秒。实验表明，我们的模型能够生成过渡平滑、内容连贯的完整歌曲。通过略微修改任务设置，它还可以实现具有变化风格提示的多歌手生成。音频样本展示在演示页面中。

局限性与伦理讨论

**局限性：**SongEditor 目前重新生成指定句子，但不提供对编辑片段的额外控制。此外，在语义层面没有显式的音轨解耦，这可以增强可解释性并改善区分度。

**伦理：**SongEditor 是一种创新工具，帮助专业音乐人或热情的爱好者创作自己的歌曲。然而，我们也充分认识到潜在的伦理风险。我们确保训练数据不侵犯任何版权，并且仅使用公有领域旋律进行推理。

致谢

本工作得到深圳市科技计划（深圳市Key Laboratory Grant No. ZDSYS20230626091302006）和深圳市科技研究基金（Fundamental Research Key Project Grant No. JCYJ20220818103001002）的支持。